

68. Un état de jeu

Quelques scènes, et un `switch` qui dit laquelle est en cours.

1. Ce qu'il faut comprendre

C'est presque toujours la forme d'un jeu : un `enum` de scènes, et un `switch`. Le reste — les ennemis, le décor — se branche dans le `case JEU`.

Le front est calculé **une fois**, en haut de la boucle, et servi à tous les cas. Le recalculer dans chaque `case` marcherait aussi — et un jour l'un des trois serait oublié.

Le score n'est écrit **que lorsqu'il change**, et l'étiquette une seule fois en entrant. Les réécrire à chaque image alourdit la boucle au point qu'elle déborde — et un `depuis` qui compte les tours avance alors trois fois moins vite que le temps réel.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1558 octets** de cartouche.

1. DES CONSTANTES NOMMÉES — SCENE

```
enum Scene { TITRE, JEU, PERDU };
```

2. UNE VARIABLE — OU

```
Scene ou = TITRE;
```

3. UNE VARIABLE — SCORE

```
uint8_t score = 0;
```

4. UNE VARIABLE — AVANT

```
uint8_t avant = 0;
```

5. UNE VARIABLE — DEPUIS

```
uint8_t depuis = 0;
```

6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    while (true) {
        uint8_t appui = bouton(START);
        uint8_t front = appui && !avant;
        avant = appui;

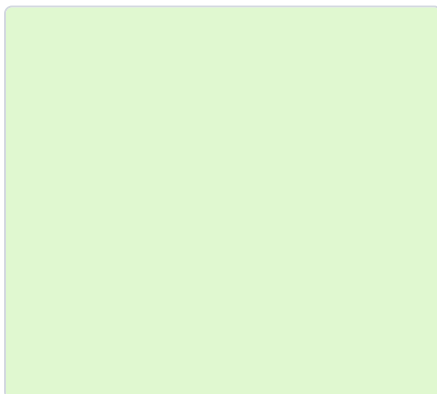
        switch (ou) {
            case TITRE:
                texte(4, 8, "APPUIE START");
                if (front) {
                    texte(4, 8, " ");
                    score = 0; depuis = 0; ou = JEU;
                    texte(2, 4, "SCORE"); // une fois, en entrant
                    nombre(9, 4, score);
                }
                break;

            case JEU:
                depuis++;
                if (depuis >= 30) { depuis = 0; score++; nombre(9, 4, score); }
                if (score >= 20) ou = PERDU;
                break;

            case PERDU:
                texte(5, 8, "PERDU");
                if (front) { texte(5, 8, " "); ou = TITRE; }
                break;
        }
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



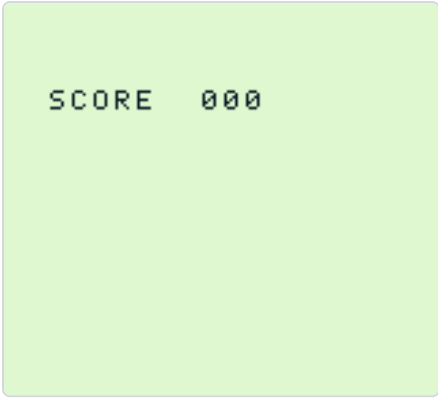
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



SCORE 000

START — l'écran a changé

5. Ce qu'on doit voir

START lance la partie, le score monte, et à 20 c'est perdu.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ on démarre sur le titre
- ✓ START lance la partie (score 3)
- ✓ le score finit par atteindre vingt (20)
- ✓ et l'on passe à « PERDU »
- ✓ START ramène au titre

Ensuite : **69. Le score, et le panneau** — niveau 9 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++